CIS 3300 Full-Stack Development Project

Build Something of Your Own

This document covers everything about the project. Use it as your reference all term.

Note. For those that are in multiple courses, please note that there may be slightly different requirements per course, including the schedule of due dates. Please read carefully and ask if you are unsure about something.

Contents

Foreword

This project exists because students asked for it.

A couple of semesters ago, students told me in end-of-semester surveys and graduation exit interviews that they wanted to work on full-stack development and build something real, something of their own. Enough of them said it, clearly enough, that I made room for it. I cut what was most often identified as “busy work” in the course so this project could exist. You are getting the benefit of that.

That is worth noticing. Feedback from students actually changes things here, and it changed this.

Some of you know people who have already done this project. Ask them what it was like. And when you are through it, tell people yourself, whether that is a student deciding what to take next semester or a survey at the end of this one.

Do Not Fret Over This

This is meant to be fun for you because it is yours.

You are building something you chose, at your own pace, and nobody is grading how impressive it turns out. Your project does not have to work. It does not have to be finished. It does not have to be original, or ambitious, or anything other than yours.

What it does need is for you to keep at it and keep me in the loop. That is the whole arrangement. I am your manager on this, which mostly means I am the person you talk to when something goes sideways, and something will go sideways for almost everyone. That is normal, it is expected, and it is not a problem as long as I hear about it.

If you find yourself stressed about this project, come see me. That is usually a sign that either the scope needs adjusting or something has gone wrong that we can fix quickly. Neither one is a big deal, and both are easier to solve early.

1. Your Mission

Build an application you actually want to build. Maybe you already have an idea you have been meaning to try. Maybe you have seen something someone else built and wanted to make your own version. Both are good places to start, and imitation is a legitimate way to learn: rebuilding something you admire will teach you more than inventing something you do not care about.

The subject is yours. What ties it together is that the project should draw on the material we cover, and that you keep it moving across the semester. It does not have to be an original idea, and it does not have to be finished by the end of the term; projects seldom are. This project is about what you build, what you learn, and what you can show for it.

You may build in any language. Java is recommended, since that is the language of this course and you will have the most support here, but the choice is yours.

2. This Does Not Have to End This Semester

Pick something you actually care about, because you may not be done with it.

If you take another course with me, you can carry this project forward. You start from where you left off, build the next layer with the new course's material, and get credit for it there. The application you build here could pick up threading, sockets, and a client-server architecture in CIS 3970, Secure Programming Principles. It could get real data structures underneath it in CIS 4020. It could go further still in special topics sections or graduate courses when those are offered.

Over two or three semesters, that adds up to something substantially more interesting than three unrelated assignments, and it is something you can show people.

This is not a requirement, and starting fresh each time is a perfectly good choice. But if you find something you want to keep working on, you can.

3. How This Works

You are the developer. I am your manager. The project is self-directed and you set its direction, but a working relationship has obligations attached: you keep your manager informed, you deliver on agreed dates, and you show real work rather than descriptions of work you intend to do.

Your project grade is determined by specifications. Each requirement is either met or not met on a stated date, and your grade follows from which ones you complete. The quality of your project

does not lower your grade. Failing to do the work does.

Layoffs are real. Failure to meet the requirements of this project can result in being laid off, which means a course grade of F. See the section Professionalism and Full-Stack Projects in the syllabus for the full range of conduct that can lead to a layoff. It extends well beyond this project.

4. Project Guidelines

- Build in any language you choose. Java is recommended but not required.
- Complete the course mapping in your proposal, so you have thought about what goes into what you are building.
- Do your own work, but take advantage of tools that exist. You may use development tools, generative AI, tutorials, and references, provided you document what you used and explain your role in the result.
- Creativity and customization count. Build something that feels like yours.
- All project materials must have a copy in your project repository, except videos hosted elsewhere. Organize the repository so a reader can find things.
- Keep every AI conversation from the start of the project. These are artifacts of your work and are submitted with your logs and final submission.
- If you want to pursue an idea that does not fit these guidelines, talk to me first.
- Ask me questions directly. Other instructors and advisors will not know the expectations for this project.

Taking Two of My Courses at Once

If you are taking two of my courses at the same time, you may build on the same idea in both. Everything else is separate. Each course gets its own work, its own logs, its own documentation, and its own presentation covering that course's portion. Some overlap in the underlying idea is expected and fine, but work submitted for one course does not count for the other. Two courses means two courses' worth of work, so plan accordingly. If this is your situation, come talk to me when you propose and we will sort out the specifics.

Continuing a Project From a Previous Course

Continuing a project from a previous course of mine is fine, and often a good idea. What matters is that you actually continue it. Where the project stood at the end of that course is your starting point, not your deliverable. Everything you are graded on here is what you build on top of it this semester. With your proposal, record that starting point: a repository tag, a commit hash, or a short note describing where the project stood when the previous course ended.

5. Project Categories and Ideas

You are proposing your own project. What follows is a set of categories to help you figure out what kind of thing you want to build.

The examples under each category are illustrations of what projects in that category tend to look like. They are not a list to pick from, though there is nothing wrong with being inspired by one. Most projects fall into one of these categories, many span two or three, and a genuinely original idea may fit none of them. On your proposal you will name the categories your project falls into, or describe your own.

The categories are worth paying attention to for one practical reason: each kind of project naturally pulls in different concepts, which helps when you fill out the course mapping.

It Does Not Have to Be Original

Nobody expects you to invent something that has never been built. Most of these ideas have been built thousands of times, and that is fine. Build a version of something you have seen and liked. Rebuild a tool you already use. Make your own take on a game you enjoy. Rebuilding something you admire will teach you more than inventing something you do not care about, and it is a legitimate way to learn that professional developers use constantly.

What makes it yours is the building and the choices you make along the way, not the novelty of the concept.

If Nothing Here Fits

If your idea does not fit any of these categories, that is not a problem. Describe it in your own words on the proposal and we will talk about it. Original ideas are welcome, they are just not required.

Choosing and Sizing

Pick something you would be a little annoyed to abandon. Motivation is the resource that runs out first, and interest is what keeps a semester project alive in week ten.

On size: if an idea feels too large, narrow it to one feature done properly rather than five features half-built. If it feels too small, add a dimension such as persistence, a second interface implementation, or a data import. Either way, we will right-size it together when I review your proposal, so do not worry about getting the scope perfect on the first attempt.

With that in mind, here are the categories.

Simulations and Models

Modeling a world of related things that behave differently. Projects here lean naturally on inheritance, polymorphism, and interfaces.

- Traffic intersection simulator with different vehicle types and signal logic
- Ecosystem or predator-prey simulation with species as a class hierarchy
- Bank or credit union model with account types, transactions, and interest rules
- Elevator dispatch simulator for a building with configurable floors and cars
- Warehouse or logistics simulation with orders, packers, and shipping methods
- Airport or train scheduling model with delays, gates, and cascading effects

Games

Rules, state, and things that interact. Strong on interfaces, polymorphism, collections, and exception handling. Console games are entirely acceptable; graphics are not required.

- Text adventure with rooms, items, and an inventory system
- Card game (blackjack, poker, rummy) with a rules engine
- Turn-based combat or role-playing game with character classes and abilities
- Tile or grid game such as minesweeper, sokoban, or a maze solver
- Roguelike dungeon crawler with procedurally generated levels

- Board game engine that supports more than one game through a shared interface

Tools and Utilities

Something that does a job. Strong on file handling, exceptions, and collections. These are the projects most likely to be genuinely useful to you afterward.

- Log file analyzer that parses, filters, and summarizes
- CSV or JSON toolkit for converting, validating, and reshaping data files
- File organizer that sorts, deduplicates, or renames by rule
- Personal budget or expense tracker with categories and reports
- Study or flashcard tool with spaced repetition and saved progress
- Backup or sync utility with conflict detection

Data-Driven Applications

Built around storing and retrieving information. Strong on database work, collections, and file handling.

- Inventory or asset management system with search and reporting
- Library, media, or collection catalog with lending or checkout tracking
- Recipe manager with ingredients, scaling, and shopping list generation
- Gradebook or course management tool
- Contact or client relationship manager
- Habit or workout tracker with history and statistics

Domain Modeling

Take a subject you already care about and model it properly. Strong on class design, interfaces, and collections.

- Music library with artists, albums, tracks, and playlist rules
- Sports statistics system with players, teams, seasons, and computed standings
- Fantasy league manager with drafting, scoring, and rosters
- Tabletop role-playing character system with races, classes, and derived statistics
- Trading card or collectible system with sets, rarity, and deck validation
- Transit or route planner modeling stops, lines, and transfers

Generative and Creative

The program makes something. Strong on recursion, algorithms, and streams.

- Procedural map, dungeon, or terrain generator
- ASCII art or fractal renderer
- Music or drum pattern generator with rule-based composition
- Maze generator and solver with multiple algorithms compared
- Text generator using Markov chains over a corpus you supply
- Puzzle generator and solver such as sudoku or crosswords

Working With Existing Software

Start from something that already exists. Strong on reading unfamiliar code, which is most of what professional development actually involves.

- Contribute a feature or bug fix to an open-source project you use
- Fork an existing tool and customize it substantially for your own needs
- Add a missing capability to an open-source application
- Write a plugin or extension for an existing platform
- Improve documentation and tests for a project, then build something on top of it
- Modernize an abandoned project by updating it to current libraries

If you contribute to a live project, the maintainers may not respond, or they may turn your contribution down. That does not hurt you here. You still read the code, still solved the problem, still did the work, and still learned something. A rejected contribution is progress.

6. Thinking About What Goes Into It: The Course Mapping

With your proposal you complete a mapping of course concepts against your project. The purpose is not coverage. The purpose is to make you think about what actually goes into building software before you start building it, and to notice the pieces your project will need.

This is not a checklist. You are not expected to use every topic, and you are certainly not expected to use every subtopic. Nobody is counting. A short honest mapping is worth more than a padded one. Mark what your project will genuinely use and leave the rest alone.

The subtopics under each heading are prompts to think with, not requirements. Use the Other row for anything your project needs that is not listed.

Table 1. *Course topics to consider*

Topic	Subtopics to consider
Classes, Objects, and Interfaces	constructors, fields and encapsulation, accessors, static versus instance members, toString and equals, interface definition and implementation, abstract classes, composition
Inheritance and Polymorphism	superclass and subclass design, method overriding, calls to super, abstract methods, dynamic dispatch, casting and type checking
Exceptions, Assertions, and Logging	try and catch, checked versus unchecked, custom exception classes, resource handling, input validation, logging
Arrays and ArrayLists	one-dimensional arrays, two-dimensional and ragged arrays, list operations, iteration idioms, sorting and searching, utility methods
File I/O (text, CSV)	reading and writing files, parsing delimited data, handling malformed input, resource management
File I/O (JSON, bytes)	JSON reading and writing, binary streams, object serialization
Database Programming	connections, parameterized queries, result handling, schema design, transactions

Topic	Subtopics to consider
Generics and Collections	generic classes and methods, maps, sets, lists, queues, iterators, comparators and ordering
Lambda Expressions and Streams	lambda syntax, functional interfaces, method references, filter and map and reduce, collectors
Recursion	base and recursive cases, tree or graph traversal, divide and conquer, backtracking
Other	anything else your project uses that is not listed above, such as concurrency, networking, GUI work, testing, or a library-specific technique

You do not have to know at proposal time exactly how each piece will appear. A reasonable plan is enough, and the mapping can change as the project develops. You update it at the end with what you actually used.

7. Schedule

These dates are fixed.

Table 2. *Project schedule*

Week	What Is Due	Format
2	Proposal	Proposal with mapping
4	Log 1	Log
6	Log 2	Log
7	Progress video	5 to 7 minute demonstration, recorded or live
10	Log 3	Log
12	Log 4	Log
16	Final submission	Deliverable and repository, presentation, documentation form

See the Tentative Schedule in Course Information for specific due dates.

Logs use the log template provided with the project materials. Use the template; do not invent your own format.

8. What Progress Looks Like

Progress is not the same as finished features. On a project you designed yourself, progress is whatever moved you closer to being able to build the thing, and that varies enormously depending on what you ran into.

Progress includes

- Learning a tool, library, or framework you needed. If you spent a week getting a database driver working and understanding how it connects, that is progress. You know something

now that you did not know before.

- Trying an approach that failed and understanding why. A dead end you can explain is worth more than a feature you copied without understanding.
- Design work: sketching your class structure, working out what objects you actually need, deciding how pieces will talk to each other.
- Setting up infrastructure: the repository, the build, the environment, the project skeleton.
- Reading and research: finding out how something is normally done before doing it.
- Getting help. If you came to me or to someone else with a problem and worked through it, that is progress and it belongs in your log.
- Writing code that works, obviously. But this is one form of progress among several, not the only one.

Progress does not include

- Nothing at all. "I have been busy" is not progress.
- Trivial work that did not move the project. Renaming files, reformatting code, or reorganizing folders is maintenance, not progress.
- Plans and promises. What you intend to do next month is not something you did.
- A statement that you made no progress. Reporting the absence of work is not a substitute for the work.
- Excuses and explanations for why nothing happened. These may be entirely true, and they still are not progress.
- Work you cannot describe. If you cannot say what you did and what came of it, there is nothing to report.

The honest test is whether you can point at something and say what it cost you and what you got out of it. If you can, that is progress, and it does not matter that it was not the thing you planned to do that month.

A log reporting no progress is a missed log. It counts exactly the same as not submitting one at all. Given how many things count as progress, and that simply coming to talk with me about a problem counts, there is no reason to end up here.

When It Is Not Going Well

Projects stall. Tools break. Semesters get busy. None of that is a problem in itself, and none of it is something you will be penalized for.

What matters is what you do about it, and when. If you are stuck, deal with it then. Come see me, send an email, ask for help. Do not wait for the next log to mention it, because the log is a report on what already happened, not a request for rescue. Handle the problem, then log what you did about it.

If you need more time, spend a token. That is what tokens are for, and using one is the professional move, not a failure.

Going quiet is the one thing that does not work. A student who comes to me in week 10 because the database piece fell apart is a student I can help. A student who says nothing until week 16 is a student I could not help, and by then the semester is gone.

Failing at something is fine. Not dealing with it is not.

9. The Progress Video

Due Week 7. Five to seven minutes. There is no written report attached to it.

This is a status demonstration, not a sales pitch and not a reflection. Show me the project on screen. Rough and unfinished is expected and fine at this stage.

What makes it count. The video must show actual project material: code, a running program, a repository, a database, something that exists. A video that only describes what you intend to build does not meet the specification.

Three ways to do it

- Record a video and share a link (OneDrive, YouTube, or your repository)
- Meet with me in person, scheduled in advance
- Meet with me by video conference, scheduled in advance

If you meet in person or by video conference, schedule it far enough ahead that it happens by the deadline. For a live session, what you submit is the date and time we met rather than a link.

Format

- Five to seven minutes, whichever way you do it
- Screen recording or screen sharing, so I can see the work
- Slides are optional. Demonstration is required. Production quality does not matter.

Suggested Structure

- **Reminder (30 seconds):** project name, what it does, who it is for
- **What you have built (3 to 4 minutes):** show it. Code, structure, repository, running features, database connections, whatever exists. This is the part I most want to see.
- **What you taught yourself (1 minute):** show a tool, library, or technique you did not know before. Demonstrate it working rather than narrating what the experience taught you.
- **What is next (1 minute):** features in progress, immediate next steps, known risks
- Seven minutes is part of the exercise. Your manager has a few minutes and wants to see what you have. Decide in advance what is worth showing rather than narrating everything you did.

It is not a video for apologies, reflections, or promises to do more in the future. It is a video demonstrating what you have done.

10. The Nine Specifications

Table 3. *The nine specifications*

#	Specification	Due	Type
1	Proposal submitted and approved	Week 2	Gate
2	Log 1 submitted, showing progress	Week 4	Ladder

#	Specification	Due	Type
3	Log 2 submitted, showing progress	Week 6	Ladder
4	Progress video submitted, demonstrating real material	Week 7	Ladder
5	Log 3 submitted, showing progress	Week 10	Ladder
6	Log 4 submitted, showing progress	Week 12	Ladder
7	Final deliverable and complete repository submitted	Week 16	Gate
8	Final presentation given, including demonstration	Week 16	Gate
9	Final documentation form submitted	Week 16	Gate

11. How Your Project Grade Is Determined

Two things are gates. Everything else sets the grade.

Gate 1: The Proposal

No approved proposal means you are laid off, and there is no project to grade, which is an F.

Gate 2: The Week 16 Submission

All three Week 16 items are required: the final deliverable and repository, the presentation, and the documentation form. Missing any one of them is an F, regardless of everything else you did.

The Grade Ladder

If both gates are cleared, your grade is set by the progress video and your logs. A log counts as met if it was submitted on time and reports real progress, or if it was covered by a token.

Table 4. *Grade ladder*

Grade	Points	Requirements
A	100	Progress video submitted, and all 4 logs met.
B	85	Progress video submitted, and 3 of 4 logs met.
C	75	Progress video submitted, and 2 or fewer logs met.
D	60	Progress video not submitted.
F	0	Any gate not met (no approved proposal, or any Week 16 item missing).

These are the only possible scores for this project. There is no partial credit within a grade. You either met the specifications for that grade or you did not.

The progress video is required for a C or higher. Without it, no number of logs will move you above a D. You have two tokens, a full week of recovery time, and three different ways to deliver it, so there is no real reason to miss it.

Note what else this means. A log you handled with a token still counts as met. Communicating and adjusting costs you a token, not a grade. Going silent is the thing that moves you down the

ladder.

What Counts as Met

- A log is met when it is submitted on time and reports real progress. Length does not matter; there is no page requirement. A log reporting no progress does not count. Use the log template.
- The progress video is met when it demonstrates actual project material. A video that only describes plans does not meet it.
- The final deliverable is met when your code and complete repository are submitted and correspond to the project you proposed.
- The presentation is met when you give it and it includes a demonstration. You are not graded on speaking skill.
- The documentation form is met when it is submitted with its fields completed.

A submission missing the parts a requirement calls for has not been made, and the specification remains unmet. This is a check that the required components are present, not a judgment of how good they are.

12. Tokens

You receive two tokens at the start of the semester. Things come up that nobody plans for, and tokens exist to cover them. Using one is the professional move. It is what you do instead of going quiet.

How tokens work

- Two tokens for the entire semester, and no additional tokens are granted.
- A token may be used on the proposal, any log, or the progress video. It is a shared pool, so what you spend early is not available later.
- Tokens cannot be applied to any Week 16 item. The term ends and there is no time to accommodate them.
- A token buys either an extension or one revision and resubmission of that item.
- A log covered by a token counts as met for the grade ladder.

Timing, which is the part that matters

- You must request a token within three days of the original deadline.
- A token moves that item's deadline to seven days after its original due date. This date is fixed.
- Requesting sooner gives you more time to do the work. Requesting later does not move the date; it only leaves you less time.
- A token applies only to the current item. Once the recovery period ends, that item is closed permanently and cannot be reopened later with a token.

Read that again. If you miss a Monday deadline and come to me that same day, you have until Sunday. If you wait until Wednesday, you still have until Sunday. Delay costs you working time, not deadline time.

13. Final Submission and Presentation

Three things are due in Week 16: your deliverable and repository, your presentation, and your documentation form. All three are required. Missing any one of them is an F, regardless of everything else you did.

On timing. Tokens cannot be applied to any Week 16 item. The term ends and there is no time to accommodate an extension. Plan accordingly.

Deliverable and Repository

Everything goes in the repository. All of it.

- All source code
- All databases, schemas, and data files
- All documents, diagrams, notes, and design work
- All AI conversations, prompts and outputs, as PDFs
- Anything else you produced for this project

Work that does not run is still submitted. Broken code, abandoned attempts, half-finished features, and dead ends all go in. They are evidence of what you did, and this project has never required that everything work. Hiding them serves no purpose.

Organize the repository so a reader can find things, and include a README that says what the project is and how to run it.

Your project does not have to be complete. It has to be the project you proposed, and it has to be real. A project that fell short of what you hoped still meets this specification. Account for what happened on your documentation form.

Presentation

Five to ten minutes, live or recorded. This is about the project, not about you.

No reflection in the presentation. Do not narrate what you learned about yourself, how you grew, or what the experience taught you. That goes on the documentation form. This is a demonstration of what you built. Showing a tool or technique you learned is welcome, because that is showing the work.

Requirements

- A demonstration is required, live or recorded. Slides are optional, but not recommended. If used, they should be limited.
- The demonstration must show your code and you navigating what you built.
- If you present live, test everything first and have a backup recording or screenshots.

Suggested Structure

- **Introduction (1 minute):** who you are, project title, what it does
- **Motivation (1 minute):** what you wanted to build and why you chose it
- **Demonstration (4 to 7 minutes):** key features, how it runs, example input and output, important design choices, interesting challenges, tools or techniques you picked up
- **Closing (30 seconds):** what you would add next

You are not graded on being a polished speaker. Speak clearly, keep the demo visible, and do not

read from a script. The specification is that you gave the presentation and demonstrated the work.

Documentation Form

Fill out the provided form. It is a form, not a report. There is no length requirement and no essay. A few lines per field is all that is wanted, and it should take fifteen or twenty minutes if you actually did the work.

Its fields cover your project name and description, final status, the updated course mapping, a few lines on what you learned and what you would do differently, resources used, AI use, and your links.

14. AI and Outside Resources

This project operates under different terms than the weekly assignments in this course. For the project, you may use generative AI and other outside resources, provided you document what you used and what your role in the result was.

Keep every AI conversation from the start of your project. These are artifacts of your work and must be submitted with your logs and in your final submission as PDFs. You are not graded on the content of these conversations. I want to see how you worked through problems, and having a record of how AI was actually used in this course matters beyond this class.

Be honest about it. There is no reason to claim you used AI only to learn something if it generated your starting code. Documented use is expected and is not penalized. Undisclosed use is a different matter and falls under the academic integrity policy in the syllabus.